The Stable Marriage Problem

Lecture Notes – Advanced Algorithms 2026, Lecture 1

These notes follow the lecture slides *The Stable Marriage Problem* by Johan M. M. van Rooij and Jesper Nederlof (*Algorithms and Networks*). They cover the same material, but with the arguments written out in full, together with the informal reasoning that usually gets spoken rather than projected. Nothing here goes beyond the content of the lecture; the short outlook in Section 7 is the only part that is explicitly non-examinable.

Contents

1 Introduction: markets that clear themselves

Most of the algorithmic problems you have met so far come with a built-in notion of what a good solution is: shortest, cheapest, largest, fastest. You write down an objective function, and then you optimise it. This lecture is about a problem where the interesting question is not *how good* a solution is, but whether it is *self-enforcing* — whether the people involved will actually go along with it once you hand it to them.

The setting is deceptively simple. There are two groups of participants of equal size. Everybody in the first group ranks everybody in the second group, and vice versa. We must pair them up. There is no global score to maximise; there is not even an obvious candidate for one, because the participants' preferences are in direct conflict with each other. What we can ask for is much weaker, and much more interesting:

> Is there a pairing that nobody has an incentive to walk away from?

A pairing with this property is called *stable*. The question of whether one always exists, and how to find it, was answered in 1962 by David Gale and Lloyd Shapley in a paper with the memorable title *"College Admissions and the Stability of Marriage"* [1]. Their answer is a short, elegant algorithm — the *deferred acceptance* or *Gale–Shapley* algorithm — which we will develop, analyse and dissect over the course of this lecture.

Why anyone cares

The problem is not a puzzle invented for a textbook. It describes a real and recurring situation: a market in which money does not do the allocating, because both sides have to agree.

The canonical example, and the one that gave the field its practical urgency, is the assignment of graduating medical students to hospital residency positions in the United States. In the 1940s this market had collapsed into chaos. Hospitals, competing for the best students, kept moving their offers earlier and earlier — eventually to two full years before graduation, at which point neither side had any real information about the other. Offers came with exploding deadlines of a few hours. Students accepted positions and then reneged when something better appeared; hospitals did the same. The market was *unravelling*.

In 1952 the profession adopted a centralised clearinghouse. Everybody submits a ranked list, a computation is performed, and the resulting assignment is binding. The crucial design question was: what computation? If the output can be undermined — if there is a student and a hospital who would both rather have each other than what the clearinghouse gave them — then they will simply arrange that privately, other participants will notice, and the clearinghouse loses its authority. Stability is exactly the property that prevents this. Remarkably, the algorithm the clearinghouse adopted in 1952 was essentially deferred acceptance, a decade before Gale and Shapley published it and proved that it works; this was only recognised much later by Alvin Roth [4].

In 2012 Lloyd Shapley shared the Nobel Memorial Prize in Economic Sciences with Alvin Roth, "for the theory of stable allocations and the practice of market design". Shapley's citation covers a number of contributions — he is at least as famous for the Shapley value in cooperative game theory — but the Gale–Shapley algorithm is prominent among them. Roth's share of the prize was for taking the theory into the field: redesigning the residency match, and later the New York and Boston public school choice systems and kidney exchange programmes. David Gale had died in 2008, and the prize is not awarded posthumously.

[figure placeholder]

A timeline / context figure for the residency match: "1940s: offers creep to 2 years before graduation, exploding deadlines, market unravels" → "1952: centralised clearinghouse (NRMP)" → "1962: Gale & Shapley explain why it works" → "2012: Nobel Prize". Could be a simple horizontal arrow with four labelled milestones.

Figure 1: The stable marriage problem was solved in practice before it was solved in theory.

[figure placeholder]

Portrait of Lloyd Shapley (Nobel Prize in Economics, 2012). Slide 2 of the deck uses the Wikimedia Commons photograph by Bengt Nyman, `http://en.wikipedia.org/wiki/File:Lloyd_Shapley_2_2012.jpg` *— check the CC licence terms and reproduce the attribution.*

Figure 2: Lloyd Shapley (1923–2016).

What this lecture will do

- Section 2 sets up the model and defines stability precisely.
- Section 3 shows why the obvious approach — repeatedly repair whatever is broken — can run forever, and why the mere *existence* of a stable matching is a genuine theorem rather than an observation.
- Section 4 presents the Gale–Shapley algorithm.
- Section 5 proves that it terminates within n^2 proposals, that its output is a perfect matching, and that this matching is stable. Together these give the existence theorem.
- Section 6 asks who benefits. The answer is sharp and slightly disturbing: the proposing side gets, simultaneously, the best partner it could possibly have in *any* stable matching, and the other side gets the worst.

A note on terminology. The classical language of "men" and "women" is historical, dating from Gale and Shapley's original framing, and we keep it because the slides, the literature and the name of the problem all use it. Mathematically nothing depends on it: the content is that there are two disjoint sides, and that matches are only ever formed between the sides and never within one. Read "students and hospitals" throughout if you prefer.

2 The model

2.1 Definitions

Definition 2.1 (Instance)**.** An instance of the *stable marriage problem* consists of a set $\mathcal{M} = \{m_1, \ldots, m_n\}$ of n men and a set $\mathcal{W} = \{w_1, \ldots, w_n\}$ of n women, together with, for every participant, a *preference list*: a strict total order on all n members of the opposite side.

We write $w \succ_m w'$ to mean "man m strictly prefers w to w'", i.e. w appears before w' on m's list, and symmetrically $m \succ_w m'$ for the women. Three features of this definition deserve to be pointed out, because all three will matter later:

- **The lists are complete.** Everybody ranks *everybody* on the other side. Nobody is unacceptable. This is what will let us prove that every participant ends up matched.
- **The lists are strict.** There are no ties, no indifference. A woman confronted with two men always has a preference. Allowing ties makes several of the questions in this lecture considerably harder (see Section 7).
- **The two sides are equally large.** With $|\mathcal{M}| \neq |\mathcal{W}|$ some participants must remain unmatched, and the definitions below need adjusting.

Definition 2.2 (Matching)**.** A *matching* is a set μ of n disjoint man–woman pairs; that is, a perfect matching in the complete bipartite graph $K_{n,n}$ with sides $\mathcal{M}$ and $\mathcal{W}$. We write $\mu(m)$ for the partner of m and $\mu(w)$ for the partner of w, so $\mu(m) = w \iff \mu(w) = m$.

There are $n!$ matchings, so brute force is out of the question for anything but tiny instances. Now the key definition.

Definition 2.3 (Blocking pair, stability)**.** Let μ be a matching. A pair $(m, w) \notin \mu$ is a *blocking pair* for μ if

$$w \succ_m \mu(m) \qquad \text{and} \qquad m \succ_w \mu(w),$$

that is, if m and w each strictly prefer the other to their assigned partner. A matching is *stable* if it admits no blocking pair.

[figure placeholder]

The anatomy of a blocking pair. Draw two matched pairs as horizontal solid edges: m–w' on top, m'–w below. Then draw the diagonal m–w as a dashed/red edge. Annotate the diagonal with two arrows: "m: $w \succ w'$" and "w: $m \succ m'$". Caption idea: both endpoints of the dashed edge want to switch, so they will.

Figure 3: A blocking pair: an unmatched couple who both prefer each other to what the matching gave them.

Intuition. A blocking pair is a *private deviation that is guaranteed to succeed.* It takes exactly two people, both of whom benefit, and neither of whom needs anybody's permission. That is why stability is the right notion of "a solution the market will accept": it is not a statement about how happy people are, but about whether the assignment can be undermined by a coalition of the smallest interesting size.

Note also how weak the requirement is. We do not ask that everyone be happy; we do not ask that the total or average satisfaction be high; we do not even ask for fairness. We ask only that no *pair* can improve on the outcome by themselves. This weakness is precisely what makes the existence theorem possible — and, as we shall see in Section 6, stable matchings can be spectacularly unfair.

Remark 2.4 (Why single-person deviations are not an issue). One could imagine also forbidding a single participant from wanting to walk away. Because our preference lists are complete, nobody is ever worse off matched than unmatched, so individual deviations never occur and stability against pairs is the whole story. In models with incomplete lists (where you may declare some partners unacceptable) one also requires *individual rationality*, and blocking pairs are defined only over mutually acceptable couples.

2.2 A worked example

Example 2.5. Take $n = 3$, with $\mathcal{M} = \{\text{Arie}, \text{Bert}, \text{Carl}\}$ and $\mathcal{W} = \{\text{Ann}, \text{Betty}, \text{Cindy}\}$, and preference lists as follows (best first):

Men	1st	2nd	3rd
Arie	Ann	Cindy	Betty
Bert	Betty	Ann	Cindy
Carl	Betty	Ann	Cindy

Women	1st	2nd	3rd
Ann	Bert	Carl	Arie
Betty	Carl	Bert	Arie
Cindy	Bert	Carl	Arie

Poor Arie: every woman ranks him last. This is a good instance to keep in mind, because it makes vivid that stability has nothing to do with fairness. Arie is universally unwanted and no algorithm can fix that; the most we can hope for is that he ends up with someone who cannot do better than him *given what everybody else is doing.*

[figure placeholder]

The instance of Example 2.5 drawn as a bipartite graph. Men in a column on the left (Arie, Bert, Carl), women on the right (Ann, Betty, Cindy). Each man's preference list written next to his node in order, likewise for the women. Optionally draw all nine potential edges very faintly to emphasise that any of the $3! = 6$ perfect matchings is a candidate.

Figure 4: The instance of Example 2.5.

Consider first the matching

$$\mu_{\text{bad}} = \{(\text{Arie}, \text{Cindy}),\ (\text{Bert}, \text{Betty}),\ (\text{Carl}, \text{Ann})\}.$$

This is *not* stable. Look at Carl and Betty. Carl is matched to Ann, but Betty is his first choice, so Betty $\succ_{\text{Carl}}$ Ann. Betty is matched to Bert, but Carl is *her* first choice, so Carl $\succ_{\text{Betty}}$ Bert. They both prefer each other to their current partners: (Carl, Betty) is a blocking pair, and in any real market these two would simply pair up and leave the mechanism looking foolish.

Now consider

$$\mu^* = \{(\text{Arie}, \text{Cindy}),\ (\text{Bert}, \text{Ann}),\ (\text{Carl}, \text{Betty})\}.$$

This one *is* stable. To check this by hand, it suffices to examine, for each man, only the women he strictly prefers to his current partner — if he is content, he cannot be half of a blocking pair.

- **Carl** has Betty, his first choice. Content.
- **Bert** has Ann, his second choice; he would rather have Betty. But Betty has Carl, whom she ranks first, so she will not entertain Bert. Not blocking.
- **Arie** has Cindy, his second choice; he would rather have Ann. But Ann has Bert, whom she ranks first. Not blocking.

No blocking pair exists, so μ^* is stable. Notice how the verification worked: every time a man wanted to trade up, the woman in question was already holding somebody she liked better. That tension — men reaching downward through their lists while women climb upward through theirs — is the engine of everything that follows.

[figure placeholder]

Side-by-side comparison of μ_{bad} (left) and μ^* (right) on the bipartite graph. On the left, highlight the blocking edge Carl–Betty in red with the two "I prefer you" arrows; on the right, no red edge. Same node layout in both panels so the eye can compare.

Figure 5: An unstable and a stable matching on the same instance. The red edge on the left is a blocking pair.

Remark 2.6 (Checking stability is easy; finding a stable matching is the problem). Given a matching, we can verify stability in $\mathcal{O}(n^2)$ time by checking all pairs (with a little care, in $\mathcal{O}(n^2)$ total even naively, since there are only n^2 pairs and each check is $\mathcal{O}(1)$ given a precomputed rank table). So the problem is squarely in the "easy to check" regime. What is not obvious is how to *find* such a matching — or whether one exists at all.

3 Two false starts

Before presenting the algorithm that works, it is worth spending time on the approach that does not. This is not just for historical colour: understanding precisely *why* the naive method fails tells us what the successful algorithm has to get right.

3.1 The soap-opera algorithm

Here is the most natural idea imaginable. A matching is bad exactly when it has a blocking pair. So: find a blocking pair, and fix it.

Algorithm 1 Greedy repair (the "soap series" algorithm)

```
start from an arbitrary matching μ
while μ has a blocking pair (m, w) do
    let m' = μ(w) and w' = μ(m)
    replace the pairs (m, w') and (m', w) by (m, w) and (m', w')
end while
return μ
```

The blocking couple gets together; their two jilted ex-partners are left over, and — this being a soap series — console themselves with each other. Every step makes two people strictly happier. Surely this must get somewhere?

It need not. The two people who were made happy are not the only ones affected: the two who were dumped are typically made *much* worse off, and they will generate new blocking pairs of their own. There is no potential function that obviously decreases, and indeed the process can cycle forever.

Example 3.1 (A cycling instance)**.** Take three men $1, 2, 3$ and three women A, B, C with the following preferences (best first):

Men	1st	2nd	3rd
1	A	C	B
2	C	A	B
3	A	B	C

Women	1st	2nd	3rd
A	2	1	3
B	3	1	2
C	1	2	3

Starting from $\{1A,\ 2B,\ 3C\}$, resolve blocking pairs in this order:

$$\{1A, 2B, 3C\} \xrightarrow{(2,A)} \{1B, 2A, 3C\} \xrightarrow{(2,C)} \{1B, 2C, 3A\}$$
$$\xrightarrow{(1,C)} \{1C, 2B, 3A\} \xrightarrow{(1,A)} \{1A, 2B, 3C\}.$$

We are back where we started, and the process repeats forever.

Let us verify the first step, and leave the rest to the reader (this is worth doing once by hand). In $\{1A, 2B, 3C\}$, is $(2, A)$ a blocking pair? Man 2 is matched to B, his last choice, and prefers A; woman A is matched to 1, and ranks 2 above 1. Yes. Resolving it marries 2 to A, leaving 1 and B — the two abandoned partners — to each other. That gives $\{1B, 2A, 3C\}$.

Intuition. The lesson is about *commitment*. In the greedy algorithm every match is provisional in a destructive way: making one couple happy actively injures another couple, and the damage propagates. What we need instead is a process with a built-in ratchet — some quantity that moves in one direction only and that therefore cannot return to a previous state. The Gale–Shapley algorithm supplies exactly that, in the form of an asymmetry between the two sides: one side only ever descends its preference list, the other only ever ascends.

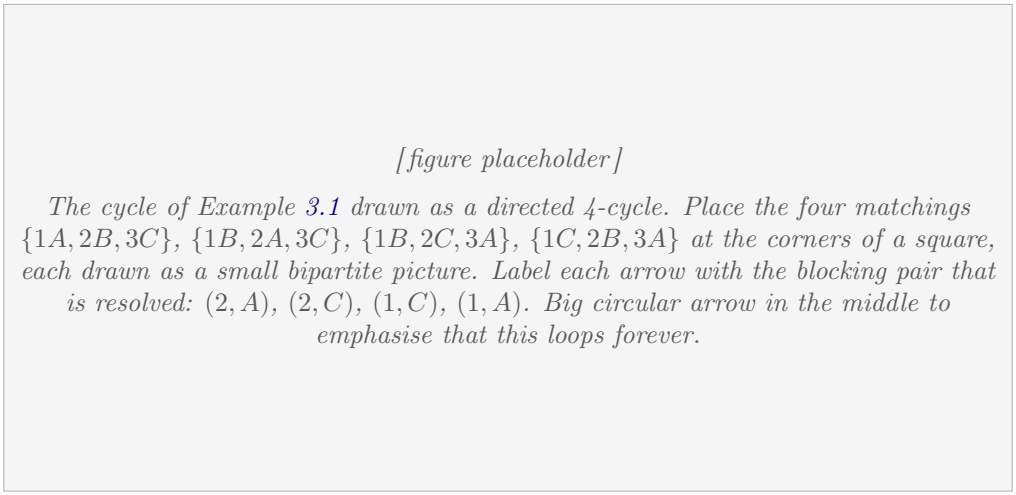

Figure 6: Greedy repair can cycle: resolving blocking pairs one at a time returns to its starting point.

Remark 3.2 (The greedy algorithm is not hopeless, only unreliable). This example shows that *some* sequences of repairs cycle. It does not show that all do. In fact Roth and Vande Vate proved that from any matching there always *exists* a finite sequence of blocking-pair resolutions leading to a stable matching, so that if at each step one picks a blocking pair at random, the process reaches stability with probability 1. That is a beautiful result, but it is not an algorithm with any useful running-time guarantee, and it does not help us prove that a stable matching exists in the first place — one has to know that already. Off-syllabus; mentioned only so that you do not draw a stronger conclusion from Example 3.1 than it supports.

3.2 Existence is not free

Here is a trap worth walking into deliberately. After staring at small examples for a while, one starts to feel that a stable matching must surely always exist — how could a market fail to have *any* outcome that nobody can undermine? But this feeling is wrong in general. It is true for the two-sided problem, and that is a theorem with content.

To see that the theorem has content, drop the two-sidedness. In the *stable roommates problem* there are $2n$ people, each of whom ranks all the others, and we pair them up without any bipartite structure. Everything else — blocking pairs, stability — is defined exactly as before.

Example 3.3 (No stable matching)**.** Four people a, b, c, d with preferences

$$a:\ b \succ c \succ d, \qquad b:\ c \succ a \succ d, \qquad c:\ a \succ b \succ d, \qquad d:\ \text{anything}.$$

The three people a, b, c form a rock–paper–scissors cycle: a prefers b, b prefers c, c prefers a; and all three rank the unfortunate d last.

Any matching pairs d with one of a, b, c and the other two with each other. Say d is matched to a, so b and c are matched. Now a is matched to his least-favourite person, so a will block with *anybody* who is willing — and c is willing, because c ranks a first and is currently matched to b. So (a, c) blocks. By the cyclic symmetry of the instance the same argument works whichever of a, b, c is stuck with d. Hence no stable matching exists.

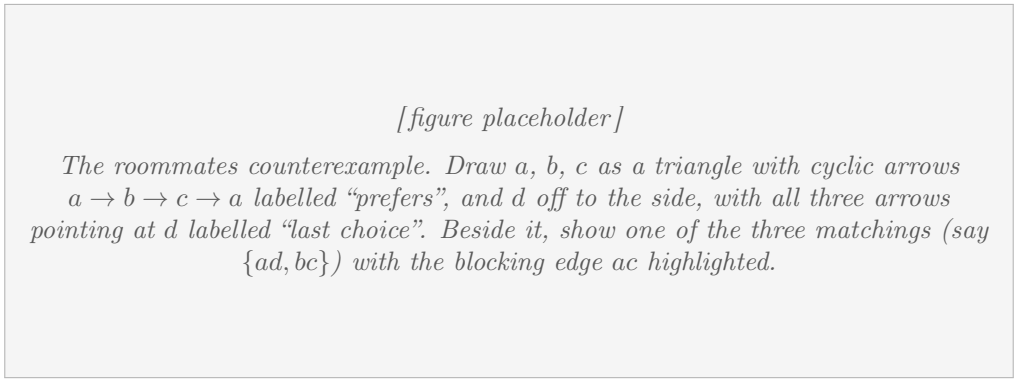

Figure 7: The stable roommates problem: an instance with no stable matching at all.

So existence genuinely uses the bipartite structure of the marriage problem. Keep Example 3.3 in mind as a control: any proof of existence for stable marriage that would also work for roommates must be wrong.

4 The Gale–Shapley algorithm

Theorem 4.1 (Gale–Shapley, 1962)**.** *There is an algorithm which, given a stable marriage instance on $n+n$ participants, outputs a stable matching using at most n^2 proposals. In particular, every instance admits a stable matching.*

The second sentence is worth pausing on. It is a purely mathematical statement — no algorithms, no computation — and yet the only proof we shall give of it is by exhibiting an algorithm and running it. This is a recurring and rather satisfying phenomenon: an efficient algorithm is often the cleanest available proof of an existence theorem.

4.1 The algorithm

The idea is *deferred acceptance*. Men propose, in order of their own preference. Women never accept anything permanently: they hold on to the best offer they have received so far and remain free to trade up later. An engagement can always be broken by the woman, never by the man.

Three points of clarification, each of which is a common source of confusion:

- **Rejection is permanent, acceptance is not.** Once w has turned m down, m will never propose to her again — he moves on down his list. But an engagement is only ever provisional; w may break it the moment somebody better comes along. This asymmetry is the whole trick.
- **"Better off with m than with her current partner"** means strictly better according to *her* preference list. She does not consider how the men feel about it, and she does not consider anybody outside the current proposal.
- **The ordering of the men is arbitrary.** The algorithm as stated picks the first free man in a fixed order, but any free man will do, and we will prove in Section 6 that the output does not depend on the choice. This is a strong and surprising statement: a completely non-deterministic procedure with a canonical answer.

Algorithm 2 Gale–Shapley (men proposing)

fix an arbitrary ordering of the men; initially everybody is free
while some man is unmatched **do**
    let m be the first unmatched man in the ordering
    let w be the highest woman on m's list to whom m has not yet proposed
    m proposes to w
    if w is free **then**
        w and m become engaged
    else if w prefers m to her current partner m'' **then**
        w and m become engaged; m'' becomes free ▷ m'' is *jilted*
    else
        w rejects m; m stays free
    end if
end while
return the set of engaged pairs

[figure placeholder]

One iteration of the algorithm, in three small panels. Panel 1: free man m with his list, arrow pointing at the next unproposed woman w; w currently engaged to m''. Panel 2: w compares m and m'' on her list. Panel 3(a): $m \succ_w m''$, so the edge to m'' is cut (dashed, m'' drops back into the pool of free men) and a new edge to m is drawn. Panel 3(b): otherwise m is rejected and returns to the pool with his pointer advanced by one.

Figure 8: A single proposal step. The woman always ends up at least as well off as before; the man's pointer only ever moves down his list.

4.2 Running the algorithm

Let us run Algorithm 2 on Example 2.5, with the men ordered Arie, Bert, Carl.

#	Proposal	Outcome	Engagements afterwards
1	Arie → Ann	free; accepts	(Arie, Ann)
2	Bert → Betty	free; accepts	(Arie, Ann), (Bert, Betty)
3	Carl → Betty	prefers Carl; **Bert jilted**	(Arie, Ann), (Carl, Betty)
4	Bert → Ann	prefers Bert; **Arie jilted**	(Bert, Ann), (Carl, Betty)
5	Arie → Cindy	free; accepts	(Arie, Cindy), (Bert, Ann), (Carl, Betty)

After five proposals everybody is matched and the algorithm stops, returning

$$\mu_{\mathcal{M}} = \{(\text{Arie}, \text{Cindy}),\ (\text{Bert}, \text{Ann}),\ (\text{Carl}, \text{Betty})\} = \mu^*,$$

the stable matching we verified by hand in Section 2.2.

Trace the dynamics once more and watch the two ratchets turn. Bert starts at the top of his list (Betty), is displaced, and moves down to Ann. Arie starts at the top of his list (Ann), is displaced, and moves down to Cindy. Meanwhile Betty goes from Bert to Carl — upward on her list — and Ann goes from Arie to Bert, also upward. *Every single event in*

the execution moves some man down and some woman up. Nothing ever moves the other way. That is the property the soap-opera algorithm lacked, and it is what we will now turn into proofs.

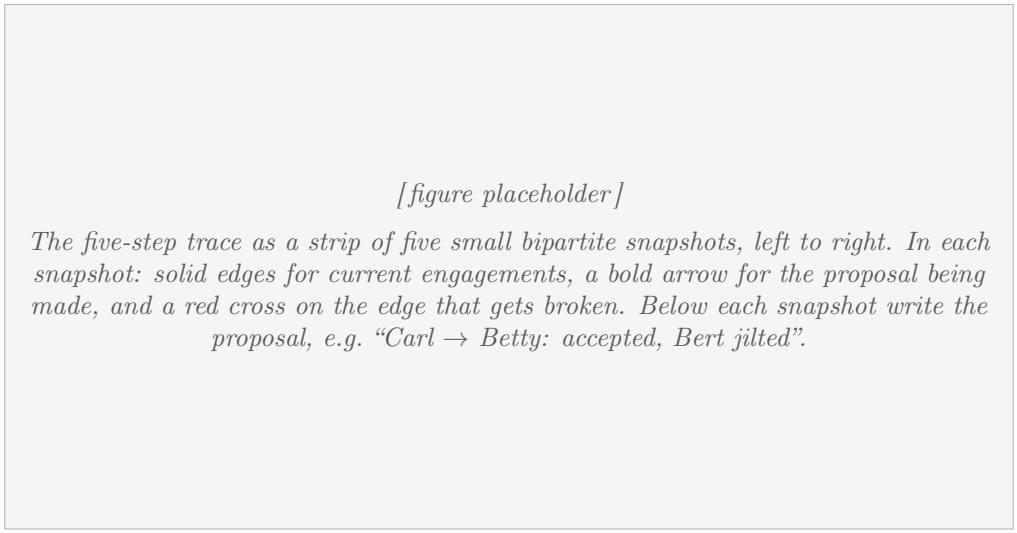

Figure 9: Execution of Gale–Shapley on Example 2.5.

Remark 4.2 (Try a different order). Run the same instance with the men ordered Carl, Bert, Arie, or let Bert propose first. The intermediate steps differ, but the final matching is identical. Theorem 6.4 explains why this is no coincidence.

5 Analysis

We now prove Theorem 4.1. There are three things to establish: the algorithm terminates (and quickly); its output is a perfect matching, so that nobody is left over; and the output is stable. Everything rests on a handful of simple invariants, which are best isolated once and then used repeatedly.

5.1 The invariants

Observation 5.1. *Once a woman is engaged, she is engaged for the rest of the execution. Her partner may change, but she never becomes free again.*

Proof. Inspect Algorithm 2. An engaged woman only ever appears in line 9, where she swaps one partner for another. There is no line in which a woman becomes free. □

Observation 5.2. *If an engaged woman's partner changes, the new partner is strictly better for her than the old one. Consequently, the sequence of partners a woman has over the course of the execution is strictly increasing in her preference order.*

Proof. A woman changes partner only in line 9, and the guard on line 8 is exactly the condition that she strictly prefers the new proposer. □

Observation 5.3. *The sequence of women to whom a fixed man proposes is strictly decreasing in his preference order.*

Proof. By line 4 he always proposes to the best woman he has *not yet* proposed to, and once proposed to, a woman is struck off his list forever. □

[figure placeholder]

The two ratchets. On the left, a man's preference list drawn as a vertical column with an arrow moving strictly downward over time (Observation 5.3); on the right, a woman's preference list with an arrow moving strictly upward (Observation 5.2). A caption such as "men get worse off, women get better off, and neither can ever go back" makes the point. This single picture explains termination, the n^2 bound, and half of the stability proof.

Figure 10: The monotonicity that makes the algorithm work.

Intuition. Observations 5.2 and 5.3 are the ratchets promised earlier. The state of the algorithm can be summarised by "how far down his list each man has got", and this vector only ever increases. That is why the process cannot cycle the way Example 3.1 did, and it immediately bounds the number of steps. Everything else in this section is bookkeeping around these two facts.

5.2 Termination and running time

Lemma 5.4 (Free men always have somewhere to go)**.** *If a man is free at some point during the execution, then there is a woman to whom he has not yet proposed. Hence line 4 is always well defined and the algorithm never gets stuck.*

Proof. Suppose for contradiction that m is free and has already proposed to all n women. By Observation 5.1, every woman who has ever received a proposal is engaged — she accepted the first offer she ever got, since a free woman always accepts, and she has stayed engaged ever since. So all n women are currently engaged.

Now count. The n women are engaged to n distinct men, so all n men are engaged. In particular m is engaged, contradicting the assumption that he is free. □

[figure placeholder]

The counting argument of Lemma 5.4. Left column: n men with m singled out and marked "free". Right column: all n women, each drawn with a solid engagement edge. The picture should make it visually absurd that n women are matched to n distinct men while one man is left over — perhaps with a large "n edges, n men, but m is not among them?" annotation.

Figure 11: A free man cannot have exhausted his list: that would mean all women are engaged, hence all men are engaged.

Lemma 5.5 (Termination and step count)**.** *The algorithm performs at most n^2 proposals and then terminates.*

Proof. By Observation 5.3, each man proposes to each woman at most once, so there are at most $n \cdot n = n^2$ proposals in total. Every iteration of the while-loop performs exactly one proposal, so there are at most n^2 iterations. By Lemma 5.4 no iteration can fail to find a target for the proposal, so the loop runs until its guard fails, i.e. until every man is matched. □

Lemma 5.6 (The output is a perfect matching)**.** *When the algorithm terminates, every man and every woman is matched, and the set of engaged pairs is a perfect matching.*

Proof. At every moment the engagements form a partial matching: each man is engaged to at most one woman and vice versa, directly by the update rules. The loop terminates only when no man is free, i.e. when all n men are engaged; since they are engaged to distinct women and there are only n women, all women are engaged too. □

Remark 5.7 (The n^2 bound is tight). It is not hard to construct instances on which almost all n^2 proposals are made; a classic family has every man with the same preference list and the women's lists arranged so that each proposal displaces somebody. Since the input itself has size $\Theta(n^2)$ — there are $2n$ lists of length n — the algorithm is optimal up to constants in the worst case, simply because reading the input already costs that much.

5.3 Implementing it in $\mathcal{O}(n^2)$ time

Lemma 5.5 bounds the number of *proposals*, which is not quite the same as bounding the running time. To get $\mathcal{O}(n^2)$ actual work, each proposal must be handled in $\mathcal{O}(1)$ time. Three small data structures suffice.

- **A stack (or queue) of free men.** Maintaining "the first unmatched man in the ordering" by scanning would cost $\mathcal{O}(n)$ per step. Instead keep the free men in any container with $\mathcal{O}(1)$ insert and extract. When a man is jilted, push him back. (Since the output is independent of the order, we may as well use whichever container is cheapest.)
- **A "next proposal" pointer per man.** Man m stores an index into his own preference list, initially 1, incremented after every proposal. By Observation 5.3 this is exactly the information needed for line 4, and it makes that line $\mathcal{O}(1)$.
- **A rank matrix for the women.** The comparison on line 8 asks "does w prefer m to m''?", which naively means searching her list: $\mathcal{O}(n)$. Precompute a table $\mathrm{rank}[w][m]$ giving the position of m on w's list. Then the test is just $\mathrm{rank}[w][m] < \mathrm{rank}[w][m'']$, in $\mathcal{O}(1)$. Building the table costs $\mathcal{O}(n^2)$, which we are paying anyway to read the input.

With these, every proposal costs $\mathcal{O}(1)$ and the total running time is $\mathcal{O}(n^2)$, matching the input size. The space usage is $\mathcal{O}(n^2)$, dominated by the preference lists and the rank matrix.

5.4 Stability

The remaining — and most important — claim is that the matching we produce cannot be undermined.

Theorem 5.8 (Correctness)**.** *The matching returned by Algorithm 2 is stable.*

Proof. Let μ be the output. Consider any man m and any woman w' with $\mu(m) = w \neq w'$; write $m' = \mu(w')$. We must show that (m, w') is not a blocking pair, i.e. that at least one of the two people involved is not interested. We distinguish two cases according to whether m ever proposed to w'.

Case 1: m never proposed to w'. The algorithm terminated with m engaged to w, and a man becomes engaged only by proposing; so m did propose to w at some point. By Observation 5.3 the women he proposed to form a prefix of his preference list, hence every woman he strictly prefers to w received a proposal from him. Since w' did not, we conclude $w \succ_m w'$: he prefers his own partner to w', and does not wish to deviate.

Case 2: m did propose to w'. Since m is not matched to w' at the end, one of two things happened: w' rejected him on the spot, or she accepted and later left him for somebody better. In the first case she was, at that moment, engaged to a man she preferred to m; in the second case she immediately afterwards became engaged to a man she preferred to m (Observation 5.2). Either way, from that moment on w' was engaged to somebody she strictly prefers to m.

By Observation 5.1 she never becomes free again, and by Observation 5.2 her partner only ever improves. Hence her final partner $m' = \mu(w')$ satisfies $m' \succ_{w'} m$: she prefers her own partner to m, and does not wish to deviate.

In both cases (m, w') fails to be a blocking pair. As m and w' were arbitrary, μ is stable. □

[figure placeholder]

The stability proof in one picture. Draw the two matched pairs m–w (top) and m′–w′ (bottom), with the potential blocking edge m–w′ dashed across the middle. Then two annotated branches: Case 1 "m never proposed to w′" with the label "⇒ $w \succ_m w'$: he is not interested", and Case 2 "m proposed and was turned down" with the label "⇒ $m' \succ_{w'} m$: she is not interested". This is the figure students remember; it is worth making it large.

Figure 12: Why no blocking pair can survive: either he never asked (so he does not want her), or he asked and was refused (so she does not want him).

Intuition. The proof is a case distinction with exactly two cases, and each is one line of reasoning. *If he never asked her, it is because he is doing better than her already* — that is Observation 5.3, the men's ratchet. *If he did ask and is not with her, it is because she found somebody better and never went back* — that is Observations 5.1 and 5.2, the women's ratchet. Every blocking pair requires both people to be interested, and the algorithm's asymmetry guarantees that at least one of them is not.

It is also instructive to see where this argument would break for the roommates problem of Section 3.2: with no two sides, there is no coherent way to say who proposes and who disposes, so neither ratchet can be set up. The proof knows about bipartiteness, as it must.

Proof of Theorem 4.1. Lemma 5.5 gives termination after at most n^2 proposals, Lemma 5.6

says that the output is a perfect matching, and Theorem 5.8 says that it is stable. Since the algorithm always succeeds, a stable matching exists for every instance. □

6 Who wins? Optimality of the proposing side

We now know a stable matching always exists and can be found in $\mathcal{O}(n^2)$ time. Two questions immediately suggest themselves, and they turn out to be the same question:

- Can there be more than one stable matching? If so, which one do we get?
- The algorithm is blatantly asymmetric — men act, women react. Does it matter who proposes?

6.1 It does matter

Example 6.1 (The battle of the sexes)**.** Take $n = 2$:

$$\text{Arie}: \ \text{Ann} \succ \text{Betty}, \qquad \text{Bert}: \ \text{Betty} \succ \text{Ann},$$

$$\text{Ann}: \ \text{Bert} \succ \text{Arie}, \qquad \text{Betty}: \ \text{Arie} \succ \text{Bert}.$$

Each man's first choice is a woman whose first choice is the other man. The preferences are as opposed as they can possibly be.

There are only two matchings here, and both are stable.

- $\mu_{\mathcal{M}} = \{(\text{Arie}, \text{Ann}), (\text{Bert}, \text{Betty})\}$: both men get their first choice, both women get their last. There is no blocking pair, because neither man wants to move.
- $\mu_{\mathcal{W}} = \{(\text{Arie}, \text{Betty}), (\text{Bert}, \text{Ann})\}$: both women get their first choice, both men get their last. Again no blocking pair, because neither woman wants to move.

Run Algorithm 2 with the men proposing: Arie asks Ann, Bert asks Betty, both are free and accept, done — we get $\mu_{\mathcal{M}}$. Run the mirror image with the women proposing and we get $\mu_{\mathcal{W}}$. Same instance, same algorithm, opposite outcomes. Every participant's fate is decided entirely by which side was given the right to propose.

[figure placeholder]

The two stable matchings of Example 6.1, side by side. Left: $\mu_{\mathcal{M}}$, with little happy faces on the men and sad faces on the women; right: $\mu_{\mathcal{W}}$, the reverse. Below, an axis labelled "men's welfare" pointing one way and "women's welfare" pointing the other, to convey that the two are in direct opposition.

Figure 13: The same instance, both stable matchings. Whoever proposes, wins.

Remark 6.2. Compare this with Example 2.5, where the men-proposing run produced μ^*. If you run the women-proposing version on that instance (do it — it takes five proposals), you get μ^* again. As we shall see in Corollary 6.8, this means Example 2.5 has exactly *one* stable matching, which is why the choice of proposer made no difference there.

So the general question "is the outcome unique?" has the answer "no". What we can hope for instead is a precise description of *which* stable matching the algorithm produces. The answer is as clean as one could wish.

6.2 Best and worst stable partners

Definition 6.3 (Stable partner)**.** A woman w is a *stable partner* of a man m if there exists *some* stable matching μ with $\mu(m) = w$. Since at least one stable matching exists (Theorem 4.1), every participant has at least one stable partner.

Write $\text{best}(m)$ for the woman highest on m's preference list among his stable partners, and $\text{worst}(w)$ for the man lowest on w's list among her stable partners.

These are definitions about the *set of all stable matchings* — a set which may be exponentially large and which we certainly do not want to enumerate. Note in particular what is *not* being claimed: there is no reason a priori why the assignment $m \mapsto \text{best}(m)$ should even be a matching. Two different men could conceivably have the same best stable partner, in which case "give everybody their best stable partner simultaneously" would be incoherent. The following theorem says that this never happens, and, moreover, that Gale–Shapley computes it.

Theorem 6.4 (Man-optimality)**.** *Every execution of Algorithm 2 — for every ordering of the men and every choice of which free man proposes next — returns the same set*

$$\mu_{\mathcal{M}} \;=\; \big\{\, (m,\ \text{best}(m)) \;:\; m \in \mathcal{M} \,\big\}.$$

Intuition. Before the proof, here is the shape of the argument. A man ends up with the first woman on his list who does not reject him, so the only way he can fail to get his best stable partner is if some stable partner of his rejects him along the way. We show that *no stable partner ever rejects anybody.*

The proof is by "consider the first time something goes wrong". Suppose some stable partner does reject somebody, and look at the very first such rejection in the execution. Being first is powerful: it means that up to this moment the algorithm has made no mistakes at all, so anything we know about the state of the algorithm can be leveraged. What we extract is enough information to build a blocking pair inside a stable matching — a contradiction.

Proof. By Theorem 5.8 the output μ of any execution is a stable matching, so $\mu(m)$ is always *a* stable partner of m. It therefore suffices to prove that $\mu(m)$ is at least as good as $\text{best}(m)$ for every m; then $\mu(m) = \text{best}(m)$ and, as this holds for every execution, all executions agree.

Main claim: during the execution, no man is ever rejected by a stable partner.

Here "rejected by w" covers both possibilities: w turning down a proposal, and w breaking off an engagement in favour of a better proposer.

Suppose the claim fails. Fix the *first* moment in the execution at which a man is rejected by a stable partner of his, say

$$w \text{ rejects } m, \text{ and } w \text{ is a stable partner of } m.$$

Because w is a stable partner of m, there is a stable matching S with $S(m) = w$. Because w rejects m, she does so in favour of some man m' whom she strictly prefers:

$$m' \succ_w m. \tag{1}$$

(If she rejected a proposal, m' is her current partner; if she broke an engagement, m' is the new proposer.)

Let $w' = S(m')$ be the partner of m' in the stable matching S. Note $w' \neq w$, since $S(w) = m \neq m'$.

Sub-claim: $w \succ_{m'} w'$.

At the moment under consideration m' has just proposed to w (or is engaged to her, having proposed earlier). By Observation 5.3, every woman whom m' strictly prefers to w has already rejected him at some earlier moment. Now suppose, for contradiction, that $w' \succ_{m'} w$. Then w' is one of those women: she has already rejected m' earlier in the execution. But w' is a stable partner of m', witnessed by S. So that earlier event was a rejection of a man by a stable partner — contradicting the choice of our moment as the *first* such event. Hence $w \succ_{m'} w'$, proving the sub-claim.

Now put the pieces together in the stable matching S, where m' is matched to w' and w is matched to m:

- $w \succ_{m'} w'$ (sub-claim): m' prefers w to his S-partner;
- $m' \succ_w m$ (1): w prefers m' to her S-partner.

So (m', w) is a blocking pair for S, contradicting the stability of S. This proves the main claim.

From the claim to the theorem. Consider the output μ and any man m. He is matched to the first woman on his list who did not reject him: by Observation 5.3 he worked strictly downward from the top, and every woman above $\mu(m)$ on his list rejected him at some point. By the main claim, none of those women is a stable partner of m. Hence $\text{best}(m)$ is not among them, so $\text{best}(m)$ is at position $\mu(m)$ or below on m's list; that is, $\mu(m)$ is at least as good for m as $\text{best}(m)$. Since $\mu(m)$ *is* a stable partner of m and $\text{best}(m)$ is by definition his best one, we get $\mu(m) = \text{best}(m)$. □

[figure placeholder]

The man-optimality proof. Suggested layout: a horizontal time axis for the execution, with a marked point "first rejection of a man by a stable partner: w rejects m in favour of m'". Everything to the left shaded as "no stable partner has rejected anyone yet". Below the axis, draw the stable matching S as two solid edges m–w and m'–w', and mark the blocking pair (m', w) as a red dashed edge, annotated with the two preferences $w \succ_{m'} w'$ and $m' \succ_w m$. Arrows connecting the two halves would show how the timeline fact yields the preference on the left.

Figure 14: The first rejection by a stable partner would create a blocking pair in a stable matching.

Corollary 6.5. *The set* $\{(m, \text{best}(m)) : m \in \mathcal{M}\}$ *is a matching, and it is stable.*

Proof. It is the output of Algorithm 2, which is a matching by Lemma 5.6 and stable by Theorem 5.8. □

This is worth savouring. Each man selfishly names the best partner he could have in any stable matching whatsoever — ignoring everybody else's constraints entirely — and these individually optimal, apparently uncoordinated demands turn out to be simultaneously satisfiable, and to form a stable matching. There is no reason to expect this from the definitions; it falls out of the algorithm. This is a good example of a theorem that is *much* easier to prove with an algorithm than without one.

Corollary 6.6. *Algorithm 2 produces a* man-optimal *stable matching: no stable matching gives any man a strictly better partner. In particular its output does not depend on the order in which the men propose.*

6.3 And now the bad news

Man-optimality sounds like an unambiguous success story. It is not, because the gain of one side is exactly the loss of the other.

Corollary 6.7 (Woman-pessimality)**.** *The matching produced by Algorithm 2 is the* worst *stable matching from the women's point of view:*

$$\mu_{\mathcal{M}} = \big\{\, (\text{worst}(w),\ w) \ :\ w \in \mathcal{W} \,\big\}.$$

That is, every woman simultaneously receives the worst partner she has in any stable matching.

Proof. Write $\mu = \mu_{\mathcal{M}}$ and suppose the statement fails: there is a woman w with $\mu(w) = m$ such that m is *not* her worst stable partner. Then some stable matching S gives her a strictly worse partner $m' = S(w)$, so

$$m \succ_w m'. \tag{2}$$

Let $w' = S(m)$ be the partner of m in S. Since $S(w) = m' \neq m$, we have $w' \neq w$.

Now, w' is a stable partner of m (witnessed by S), and by Theorem 6.4 the woman $w = \mu(m)$ is his *best* stable partner. As $w' \neq w$, this gives

$$w \succ_m w'. \tag{3}$$

But now look at S, in which m is matched to w' and w is matched to m'. By (3) m prefers w to his S-partner, and by (2) w prefers m to her S-partner. So (m, w) is a blocking pair for S, contradicting the stability of S. □

Intuition. The mechanism behind this is easier to feel than to state. The men-proposing algorithm has the men working *down* their lists, stopping as early as possible; it has the women working *up* their lists, but only among the men who happen to knock on their door. A woman can never reach out to a man she prefers; she can only wait. So she ends up with the best of a set of offers that was itself curated by the men's optimisation. Symmetrically, in the women-proposing algorithm every woman gets her best stable partner and every man his worst.

More generally, the interests of the two sides are exactly opposed across the set of stable matchings: whenever one stable matching is better than another for some man, it is worse for the woman involved. (The set of all stable matchings in fact forms a distributive lattice, with $\mu_{\mathcal{M}}$ and $\mu_{\mathcal{W}}$ as its two extreme elements. That structure is beyond this lecture, but it is the right mental picture: a whole ordered family of compromises stretching between the two algorithmic extremes.)

[figure placeholder]

The proof of Corollary 6.7. Two panels. Left panel, the Gale–Shapley output μ: edge m–w, annotated "w is m's best *stable partner (Theorem 6.4)". Right panel, the hypothetical stable matching S: edges m–w' and m'–w, with the red dashed blocking edge m–w across it and the two labels $w \succ_m w'$ and $m \succ_w m'$. An arrow from the left panel to the right showing where $w \succ_m w'$ comes from.*

Figure 15: If some woman could do worse in a stable matching than Gale–Shapley gives her, that matching would not be stable.

[figure placeholder]

Optional but useful mental picture: the set of all stable matchings drawn as a lattice diagram (a diamond-ish Hasse diagram), with $\mu_{\mathcal{M}}$ at the top labelled "best for all men / worst for all women" and $\mu_{\mathcal{W}}$ at the bottom labelled "worst for all men / best for all women", and a few unnamed compromise matchings in between. Mark clearly that only the two extremes are computed by the algorithms of this lecture.

Figure 16: Between the two extremes computed by Gale–Shapley there may be many other stable matchings.

Corollary 6.8 (A uniqueness test)**.** *If the men-proposing and women-proposing executions return the same matching, then it is the only stable matching of the instance.*

Proof. By Theorem 6.4 and its mirror image, $\mu_{\mathcal{M}}$ gives every man his best stable partner and $\mu_{\mathcal{W}}$ gives every man his worst (by Corollary 6.7 applied with the roles of the sides exchanged). If $\mu_{\mathcal{M}} = \mu_{\mathcal{W}}$, then for each man his best and worst stable partners coincide, so he has exactly one stable partner, and the stable matching is unique. □

This explains the observation made after Example 6.1: in Example 2.5 both executions return μ^*, so μ^* is the unique stable matching of that instance — which is why the choice of proposer was invisible there, and why running the algorithm with a different ordering of the men changed nothing.

6.4 What this means in practice

The theory has an immediate design consequence, and it is not a subtle one: *choosing who proposes is choosing who wins.* A clearinghouse is not a neutral computational device; it makes a distributional decision, and the mathematics makes that decision unusually explicit.

This was not merely theoretical for the residency match. The algorithm the NRMP had used since 1952 was, in the terms of this lecture, hospital-proposing: it produced the

hospital-optimal, and hence applicant-pessimal, stable matching. Once Roth's analysis made this visible, the direction of the algorithm became a subject of open dispute, and in the 1998 redesign by Roth and Peranson the match was switched to applicant-proposing. (The real system also has to handle couples who want positions in the same city, programmes with many positions rather than one, and applicants who rank only part of the list — so it is not literally Algorithm 2. But the core is deferred acceptance, and the choice of which side proposes is exactly the choice analysed above.)

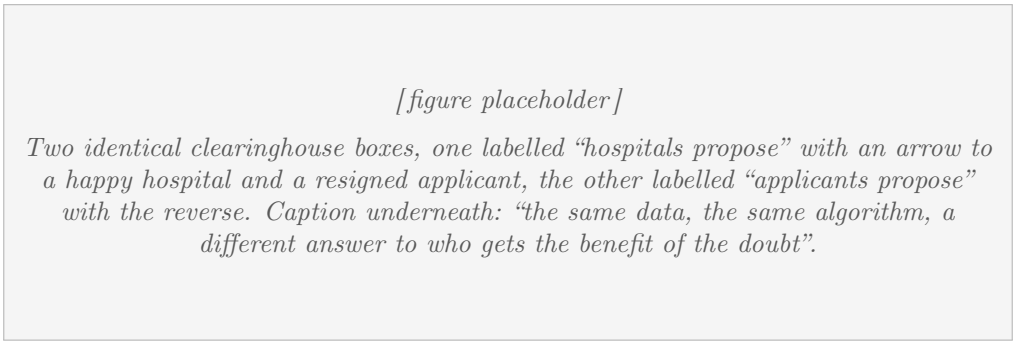

Figure 17: The direction of proposal is a policy decision, not a technical detail.

A second practical question follows immediately, and we only pose it here: if the mechanism systematically disadvantages the receiving side, do participants have an incentive to *lie* about their preferences? The answer, in brief, is that the proposing side never benefits from lying, but the receiving side sometimes does. Exercise 7 works out the smallest example.

7 Summary and outlook

What we proved

- Every stable marriage instance has a stable matching, and one can be found in $\mathcal{O}(n^2)$ time by the Gale–Shapley algorithm (Theorem 4.1). Since the input has size $\Theta(n^2)$, this is optimal.
- The algorithm is highly non-deterministic — any free man may propose next — yet its output is completely determined by the instance (Theorem 6.4).
- That output is extreme in both directions at once: every man gets the best partner he has in any stable matching, and every woman simultaneously gets the worst (Corollary 6.6 and Corollary 6.7).
- Existence is genuinely two-sided: for the stable roommates problem, where the bipartite structure is absent, stable matchings need not exist at all (Example 3.3).

Where the subject goes next (non-examinable)

Stable matching turned out to be one of those rare problems that stays tractable under a remarkable amount of generalisation, while a few natural variants fall off a cliff into NP-hardness. A sampler:

- **Unequal sides and incomplete lists.** If participants may declare some partners unacceptable, stable matchings still exist and are still found by essentially the same

algorithm, but need no longer be perfect. The *rural hospitals theorem* says something striking: the set of participants who go unmatched is the same in *every* stable matching. Unpopular rural hospitals cannot fix their staffing problem by lobbying for a different stable outcome.

- **Many-to-one (hospitals/residents).** Hospitals have capacities. Everything in this lecture goes through with the obvious modifications; this is the version actually deployed in residency matching and in school choice systems.
- **Ties (indifference).** If preference lists may contain ties, one must distinguish weak, strong and super-stability. Some of the resulting problems remain polynomial; finding a *maximum* weakly stable matching becomes NP-hard.
- **Forbidden pairs, group constraints, couples.** Many such variants remain solvable; the couples version used by the real NRMP can fail to admit a stable matching, and is NP-hard in general.
- **Strategy.** Men-proposing Gale–Shapley is *strategy-proof for the men*: no man can ever gain by misreporting his list. No mechanism can be strategy-proof for both sides at once, and the receiving side can indeed gain by lying (Exercise 7).
- **Stable roommates.** Despite Example 3.3, Irving gave an $\mathcal{O}(n^2)$ algorithm that finds a stable matching or correctly reports that none exists. Adding indifference to the roommates problem makes deciding existence NP-complete.
- **How many stable matchings can there be?** Exponentially many: instances with $\Omega(2.28^n)$ stable matchings are known, building on a construction of Irving and Leather. An upper bound of the form c^n for an absolute constant c was open for decades and was finally proved by Karlin, Oveis Gharan and Weber in 2018 [6].

8 Exercises

Exercise 1. Verify the remaining three steps of the cycle in Example 3.1: check in each case that the indicated pair really is blocking, and that resolving it gives the stated matching. Then find the (unique?) stable matching of that instance by running Gale–Shapley.

Exercise 2. Run the women-proposing version of Gale–Shapley on Example 2.5 and confirm that it returns μ^*. What does Corollary 6.8 let you conclude?

Exercise 3. Give an instance with $n = 4$ that has at least four stable matchings. *Hint:* take two disjoint copies of Example 6.1 and make each participant prefer everybody in their own copy to everybody in the other. Generalise to show that there are instances with at least $2^{n/2}$ stable matchings.

Exercise 4. Construct a family of instances on which the men-proposing algorithm makes $\Omega(n^2)$ proposals, showing that the bound of Lemma 5.5 is tight up to a constant factor.

Exercise 5. Where exactly does the proof of Theorem 5.8 use that the preference lists are strict (no ties)? Where does it use that they are complete?

Exercise 6. Prove directly from the definitions — without invoking Theorem 6.4 — that if μ and ν are stable matchings and some man strictly prefers μ to ν, then his partner in μ strictly prefers ν to μ. (This is the "opposed interests" statement that underlies the lattice structure.)

Exercise 7. Consider Example 6.1, and allow participants to submit lists that declare some partners unacceptable. Suppose Ann submits the list "Bert" only, and everybody else reports truthfully. Run the men-proposing algorithm on the reported preferences and check that Ann ends up with Bert — her true first choice, and strictly better than what truthful reporting gave her. Explain in one sentence why this trick is available to the receiving side but not to the proposing side.

Exercise 8. Implement Gale–Shapley as described in Section 5.3 and verify empirically, on random instances with $n = 1000$, roughly how many proposals are actually made and how good the men's and women's average partner ranks are. (The behaviour on random instances is strikingly asymmetric; the men do about $\ln n$ times better than the women.)

References

[1] D. Gale and L. S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962.

[2] D. E. Knuth. *Mariages stables et leurs relations avec d'autres problèmes combinatoires.* Les Presses de l'Université de Montréal, 1976.

[3] D. Gusfield and R. W. Irving. *The Stable Marriage Problem: Structure and Algorithms.* MIT Press, 1989.

[4] A. E. Roth. The evolution of the labor market for medical interns and residents: a case study in game theory. *Journal of Political Economy*, 92(6):991–1016, 1984.

[5] A. E. Roth and M. Sotomayor. *Two-Sided Matching: A Study in Game-Theoretic Modeling and Analysis.* Cambridge University Press, 1990.

[6] A. R. Karlin, S. Oveis Gharan and R. Weber. A simply exponential upper bound on the maximum number of stable matchings. In *Proceedings of STOC 2018*, pages 920–925.